

CSS4005 — Cloud Infrastructure Construction

Lecture 6: AWS IAM and Cloud Security

Nauruzbayeva Farikha · Lesson 6



Recap — Lesson 5 (AWS Storage: EBS and S3)

- When would you pick `gp3` over `io2` for an EBS volume? (`gp3` is the general-purpose default; `io2` is for sustained high-IOPS, latency-sensitive databases — and costs more)
- What does an EBS snapshot give you that the volume itself doesn't? (a point-in-time, incremental backup stored in S3 that survives the volume and can restore into another AZ)
- You overwrite `report.pdf` in a bucket with versioning on. Is the old one gone? (no — it becomes a non-current version; a lifecycle rule decides when it's really deleted)
- What is S3 Block Public Access for? (an account/bucket-level guardrail that overrides any policy or ACL trying to make data public)

Today's agenda

- ☐ Shared responsibility — which half of security is yours
- ☐ The root user and why you lock it away
- ☐ IAM building blocks: users, groups, roles, policies
- ☐ Reading and writing policy JSON
- ☐ How AWS decides: allow, deny, implicit deny
- ☐ Least privilege, worked on a real S3 example
- ☐ Roles & temporary credentials instead of access keys
- ☐ Detective tools, network guardrails, encryption
- ☐ Today's practice

The scenario

A student team builds their project on one AWS account. To "make it work", everyone logs in as root, and an access key with `AdministratorAccess` is pasted into `config.js` and pushed to a public GitHub repo.

Eleven minutes later, a bot has found the key. By morning there are 40 GPU instances mining crypto in a region nobody has ever opened — and a bill with four digits on it.

Nothing in AWS was "hacked". Every request was allowed. Today is about making sure the right requests are the only ones that are.

Block 1

Who is responsible for what

Shared responsibility model

AWS: security **of** the cloud

- Data centers, hardware, power
- Hypervisor and host OS
- Global network and regions/AZs
- Managed-service internals

You: security **in** the cloud

- Who can log in, and what they can do (**IAM**)
- Security groups, NACLs, routes
- Guest OS patching on EC2
- Your data: encryption, backups, public/private

The line moves with the service: on EC2 you patch the OS; on RDS AWS patches the database engine; on S3 you only manage access and data. IAM is **always** on your side of the line.

The root user

- Created with the account — the email address you signed up with.
- Can do **everything**, including closing the account and changing billing. IAM policies do **not** restrict it.
- Daily work as root = every mistake and every leak is unlimited.

Root checklist — do this on day one

1. Turn on **MFA** for root
2. Delete root **access keys** (there should be none)
3. Create an admin identity for daily work
4. Log out of root and keep its credentials offline

Block 2

IAM building blocks

Four nouns

	What it is	Credentials
User	A long-lived identity for one person or app	Password and/or access keys
Group	A collection of users — attach policies once	None (not an identity)
Role	An identity <i>assumed</i> by someone/something trusted	Temporary (STS)
Policy	JSON document that allows or denies actions	—

Rule of thumb: attach policies to **groups** and **roles**, not to individual users. When someone changes team, you move them between groups instead of rewriting permissions.

Identity vs resource-based policies

Identity-based

Attached to a user, group or role. "**This identity** may do X on Y."

No `Principal` — the principal is whoever it's attached to.

Resource-based

Attached to a resource — e.g. an **S3 bucket policy**, a KMS key policy. "**Who** may do X on **this resource**."

Must name a `Principal` .

```
{
  "Effect": "Allow",
  "Principal": { "AWS": "arn:aws:iam::111122223333:role/web-app" },
  "Action": "s3:GetObject",
  "Resource": "arn:aws:s3:::course-data/public-assets/*"
}
```

Within one account, an allow in *either* place is enough (unless something denies it).

Anatomy of a policy

```
{
  "Version": "2012-10-17",
  "Statement": [{
    "Sid": "ReadReports",
    "Effect": "Allow",
    "Action": ["s3:GetObject"],
    "Resource": "arn:aws:s3:::course-data/reports/*",
    "Condition": {
      "Bool": { "aws:MultiFactorAuthPresent": "true" }
    }
  }]
}
```

- **Version** — always `2012-10-17` (the policy language version, not a date you edit)
- **Effect** — `Allow` or `Deny` · **Action** — `service:Operation`
- **Resource** — an **ARN**; `*` means "everything", so be careful
- **Condition** — optional extra rules: MFA, source IP, prefix, tags...

ARNs — how you point at things

```
arn:aws:s3:::course-data          ← the bucket
arn:aws:s3:::course-data/*        ← every object in it
arn:aws:s3:::course-data/dev/*    ← objects under dev/
arn:aws:iam::111122223333:role/web-app
arn:aws:ec2:eu-central-1:111122223333:instance/i-0abc123
```

- Format: `arn:partition:service:region:account-id:resource`
- S3 bucket ARNs have **no region and no account** — bucket names are global.
- The bucket and its objects are **different resources**: `s3:ListBucket` targets the bucket, `s3:GetObject` targets objects.

Most "my policy looks right but I get AccessDenied" bugs on S3 are this: `GetObject` on `arn:aws:s3:::bucket` (missing `/`), or `ListBucket` on `bucket/*`.

Block 3

How AWS decides

Policy evaluation, simplified

1. Default

Everything is **denied**

(implicit deny)

2. Allow

An applicable **Allow**
overrides the default

3. Explicit deny

Any matching **Deny** **beats**
every allow

Explicit Deny > Allow > Implicit Deny

The full logic also includes Organizations SCPs, permission boundaries and session policies — those can only *narrow* permissions, never add them.

Worked evaluation

User `dana` is in group `developers`, which allows `s3:*` on `course-data/*`. Dana also has an inline policy:

```
{ "Effect": "Deny", "Action": "s3:*",  
  "Resource": "arn:aws:s3:::course-data/dev/secrets/*" }
```

Request	Result	Why
<code>PutObject dev/app.zip</code>	 Allow	group allow matches, no deny
<code>GetObject dev/secrets/db.env</code>	 Deny	explicit deny wins over the allow
<code>ec2:DescribeInstances</code>	 Deny	nothing allows it — implicit deny

Quick check

An admin attaches `AdministratorAccess` to a user who already has an explicit `Deny` on `iam:*`. Can that user create new IAM users?

No. An explicit deny always wins, no matter how broad the allow is.

Block 4

Least privilege

Least privilege

- Grant **only** the actions and resources a job actually needs.
- Start from nothing and add — not from `*` and remove.
- Scope three things: **actions** (verbs), **resources** (ARNs), **conditions** (when/from where).
- Review over time: unused permissions are attack surface.

AWS managed policies like `AmazonS3FullAccess` are a fast start, but they're written for *every* customer. For a real workload, write a customer-managed policy scoped to your bucket.

Worked example — read-only on one prefix

Goal: the reporting app may **list and read** `course-data/reports/` — nothing else.

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "ListReportsOnly",
      "Effect": "Allow",
      "Action": "s3:ListBucket",
      "Resource": "arn:aws:s3:::course-data",
      "Condition": { "StringLike": { "s3:prefix": ["reports/*"] } } },
    {
      "Sid": "ReadReports",
      "Effect": "Allow",
      "Action": "s3:GetObject",
      "Resource": "arn:aws:s3:::course-data/reports/*" }
  ]
}
```

Why it has two statements

- `ListBucket` is a **bucket** action → resource is the bucket ARN, and the `s3:prefix` condition limits *which folder* can be listed.
- `GetObject` is an **object** action → resource is the object ARN pattern `reports/*`.
- No `PutObject`, no `DeleteObject` → read-only by omission (implicit deny).
- No other bucket is mentioned → every other bucket is implicitly denied.

Test both sides: the allowed actions **must** work, and the forbidden ones **must** fail. A least-privilege policy you haven't tried to break is just a guess.

Block 5

Roles, temporary credentials & keys

Long-term keys are a liability

- An access key (`AKIA...`) works **until someone deletes it** — from any computer on the internet.
- Keys leak through: Git commits, screenshots, `.env` files in Docker images, Slack messages, CI logs.
- Bots scan public GitHub continuously; a pushed key is usually found in **minutes**.

Deleting the commit is **not** enough — it's still in the history and in every clone/fork (remember `git reflog` and `git log -p`). If a key is pushed: **deactivate and rotate it first**, then clean history.

Roles: borrow permissions, don't own them

- A role has a **trust policy** (who may assume it) and **permission policies** (what it can do).
- Assuming a role calls **STS** and returns temporary credentials: access key (ASIA..) + secret + **session token**, expiring after 15 min – 12 h.
- Used by: EC2 instances, Lambda, CI pipelines (OIDC), people switching into an admin role, cross-account access.

```
{ "Effect": "Allow",  
  "Principal": { "Service": "ec2.amazonaws.com" },  
  "Action": "sts:AssumeRole" }
```

↑ a trust policy: "EC2 may assume this role"

EC2 instance profiles — no keys on the server

✗ Instead of this

```
# on the EC2 instance
aws configure
# paste AKIA... key
# key now lives in ~/.aws/credentials
```

✓ Do this

1. Create role `app-s3-reader` (trust: EC2)
2. Attach the least-privilege S3 policy
3. Attach the role to the instance (**instance profile**)

```
aws sts get-caller-identity
# arn:aws:sts:...:assumed-role/app-s3-reader/i-0abc...
```

The SDK/CLI fetches short-lived credentials from the instance metadata service and rotates them automatically. Nothing to leak, nothing to rotate by hand.

Human access hygiene

- **MFA** on every human identity — and on root, first.
- Prefer **IAM Identity Center (SSO)** for people: short-lived sessions, no access keys on laptops.
- If you must have access keys: one per user, rotate them, delete unused ones (the console shows "last used").
- **Never** commit keys. Add `.env` , `*.pem` , `credentials` to `.gitignore` ; turn on GitHub **secret scanning** / **push protection**.
- Require MFA for sensitive actions with the `aws:MultiFactorAuthPresent` condition.

Quick check

Your Python app on EC2 needs to upload files to S3. A teammate suggests putting an access key in the app's environment variables. What do you do instead, and why is it safer?

Attach an IAM role (instance profile) with a policy scoped to that bucket. The app gets temporary, auto-rotated credentials — there's no long-term key to leak.

Block 6

Detect, filter, encrypt

Tools that tell you what's going on

Tool	Answers the question
IAM Policy Simulator	"Would this identity be allowed to do X on Y?" — test before you deploy
IAM Access Analyzer	"What is shared outside my account?" and "which permissions are unused?"
CloudTrail	"Who did what, when, from where?" — an audit log of API calls
Credential report / last accessed	"Which users, keys and permissions are stale?"

CloudTrail's event history (last 90 days of management events) is on by default. When something mysterious happens to your infrastructure, it's the first place to look.

Network guardrails — back to week 4

	Security group	Network ACL
Attached to	ENI / instance	Subnet
Rules	Allow only	Allow and deny
State	Stateful — replies allowed automatically	Stateless — return traffic needs its own rule
Evaluation	All rules together	In number order, first match wins

IAM decides **who may call the AWS API**. Security groups and NACLs decide **which packets reach your instances**. You need both — an open port 22 to `0.0.0.0/0` is a hole no IAM policy can close.

Encryption at rest with KMS

- **AWS KMS** manages encryption keys; services like S3, EBS and RDS use them transparently.
- S3 encrypts new objects by default (SSE-S3). EBS encryption can be turned on by default per region.
- With a **customer-managed KMS key**, the key policy is another access check: to read the data you need permission on the object **and** on `kms:Decrypt`.
- Encryption protects against lost disks and snapshots — it does **not** help if IAM lets the wrong person read the data.

Common breach checklist

- ☐ Root has MFA and no access keys
- ☐ No humans use long-term keys where SSO/roles work
- ☐ No `"Action": "*", "Resource": "*"` outside a break-glass admin role
- ☐ EC2/Lambda use roles, not embedded keys
- ☐ S3 Block Public Access on, unless a bucket is public on purpose
- ☐ No SSH/RDP/DB ports open to `0.0.0.0/0`
- ☐ CloudTrail reviewed; billing/budget alarm configured
- ☐ Secrets in Git history rotated, not just deleted

Block 7

Today's practice

Today's practice — least privilege, proven

Practice 06 has two tracks — pick one:

- **Track A (AWS):** admin group, a `developer` user limited to one S3 prefix, an EC2 role reading S3 without keys, MFA, Policy Simulator.
- **Track B (no AWS):** the same policies against a local AWS emulator (moto) with permission checks turned on.

Both tracks end with a **policy debugging** task.

```
aws s3 cp hello.txt \
s3://course-data/dev/hello.txt --profile dev
# upload: ./hello.txt to s3://course-data/dev/hello.txt

aws s3 cp hello.txt \
s3://course-data/prod/hello.txt --profile dev
# An error occurred (AccessDenied) ...
```

Your report must show **both**: what's allowed works, and what isn't allowed fails.

By the end of this lesson, you should be able to

- ☐ Split a security task between AWS and you using the shared responsibility model
- ☐ Explain when to use a user, group, role, or policy
- ☐ Read a policy and predict allow / explicit deny / implicit deny
- ☐ Write a least-privilege S3 policy scoped to one prefix
- ☐ Replace access keys on EC2 with an instance profile role
- ☐ Name the tool for: testing a policy, auditing API calls, finding public sharing

Before next lecture

- ☐ Finish Practice 06 and submit your report
- ☐ Track A: run the cleanup section — delete test users, keys, roles, instances
- ☐ Check your account: root MFA on, no root access keys

Next week's databases will live in private subnets with security groups and IAM from today — bring both mental models.

Next lecture

AWS Databases: Amazon RDS — managed relational databases, placed privately and accessed with least privilege.